Can you show me how to code the selection for each cycle and the inputs for cars passing

```
None
@route
def simulation_setup(state: State) -> Page:
    """Let the user choose the number of cycles and cars
passing."""
    return Page(state, [
        Header("Simulation Setup"),

        "How many cycles?",
        SelectBox("cycles", ["5", "10"], "5"),

        "How many cars can pass per direction each cycle? (1-100)",
        TextBox("cars_passing", str(state.cars_passing)),

        Button("Start Simulation", "run_cycle"),
        Button("Home", "index")
    ])
```

The `SelectBox` lets users choose **5 or 10 cycles**. The `TextBox` collects the **cars passing per direction each cycle**. Clicking **Start Simulation** sends both values to your existing `run_cycle` route, which validates and saves them.

write a loop that removes cars without letting counts go below zero and the code for continuing cycles and stopping at the limit?

Put this inside your existing `run_cycle` route, **after the code that saves the simulation settings**:

```
None
# Ask for settings if a simulation has not started.
if state.max_cycles == 0:
    return simulation_setup(state)
```

```
# Do not run another cycle after reaching the limit.
if state.current_cycle >= state.max_cycles:
    return simulation_results(state)

# Remove cars from every direction.
for direction in state.directions:
    direction.cars = direction.cars - state.cars_passing

    if direction.cars < 0:
        direction.cars = 0

# Record the completed cycle.
state.current_cycle = state.current_cycle + 1
state.traffic_history.append(total_cars(state))

find_busiest(state)
calculate_clearance(state)

# Show results when the final cycle is finished.
if state.current_cycle >= state.max_cycles:
    return simulation_results(state)

# Otherwise, let the user run another cycle.
content = [
    Header("Traffic Simulation"),
    "Cycle " + str(state.current_cycle)
    + " of " + str(state.max_cycles),
    "Cars waiting: " + str(total_cars(state))
]

for description in traffic_summary(state):
    content.append(description)

content.append(Button("Next Cycle", "run_cycle"))
content.append(Button("View Summary", "simulation_results"))
```

```
return Page(state, content)
```

Indent this entire section **four spaces** so it stays inside the function. The **Next Cycle** button calls `run_cycle` again using the saved settings.

from dataclasses import dataclass from drafter import (Button, CheckBox, Header…

start simulation no longer works

Your `run_cycle` function overwrites the user's settings with `cars_passing = 5` and `cycles = ""`. The lines that save settings are also indented after a `return`, so they never run.

Replace your **entire `run_cycle` route** with this:

```
None
@route
def run_cycle(
    state: State,
    cycles: str = "",
    cars_passing: int = 5
) -> Page:
    """Start a simulation or continue using saved settings."""

    # Save settings only when starting a new run.
    if cycles != "":
        if cycles not in ["5", "10"]:
            return Page(state, [
                "Choose 5 or 10 cycles.",
                Button("Back to Setup", "simulation_setup")
            ])

        if cars_passing < 1 or cars_passing > 100:
            return Page(state, [
                "Cars must be between 1 and 100.",
```

```
            Button("Back to Setup", "simulation_setup")
        ])

    state.max_cycles = int(cycles)
    state.cars_passing = cars_passing
    state.current_cycle = 0
    state.traffic_history = [total_cars(state)]

if state.max_cycles == 0:
    return simulation_setup(state)

if state.current_cycle >= state.max_cycles:
    return simulation_results(state)

# Remove cars without going below zero.
for direction in state.directions:
    direction.cars = direction.cars - state.cars_passing

    if direction.cars < 0:
        direction.cars = 0

state.current_cycle = state.current_cycle + 1
state.traffic_history.append(total_cars(state))
update_analysis(state)

if state.current_cycle == state.max_cycles:
    return simulation_results(state)

content = [
    Header("Traffic Simulation"),
    "Cycle " + str(state.current_cycle)
    + " of " + str(state.max_cycles),
    "Total cars waiting: " + str(total_cars(state)),
    "Busiest direction: " + state.busiest_direction
]
```

```
    for description in traffic_summary(state):
        content.append(description)

    content.append(Button("Next Cycle", "run_cycle"))
    content.append(Button("View Summary", "simulation_results"))
    content.append(Button("Home", "index"))

    return Page(state, content)
```

The defaults belong in the **function heading**, so Next Cycle can reuse saved settings without submitting new inputs.

I tested this correction with your code: starting, continuing, stopping at the cycle limit, and rejecting invalid capacity all passed.

its just resetting to 5 every time i put a number in for cars passing each direction

In the file you sent, these lines inside `run_cycle` force the value back to 5:

```
None
cars_passing = 5
cycles = ""
```

**Delete both lines.** The beginning of the function should look like this:

```
None
@route
def run_cycle(
    state: State,
    cycles: str = "",
    cars_passing: int = 5
) -> Page:
    """Start a simulation or continue with saved settings."""
```

```
    if cycles != "":
        if cars_passing < 1 or cars_passing > 100:
            return Page(state, [
                "Enter a number between 1 and 100.",
                Button("Back to Setup", "simulation_setup")
            ])

        state.max_cycles = int(cycles)
        state.cars_passing = cars_passing
        state.current_cycle = 0
        state.traffic_history = [total_cars(state)]
```

Keep the rest of the simulation code below that. The `= 5` **in the function heading** is only a default; it does not overwrite a submitted number.

If you already deleted those two lines, paste your **current `run_cycle` function**—the version I have would no longer explain the reset.

from dataclasses import dataclass from drafter import (Button, CheckBox, Header…

this is the code

Found it: **the lines that save your settings are indented underneath `return`**, so they never run. The saved passing capacity stays at 5, and the program returns to setup.

Replace your **entire `run_cycle` function, including its `@route` line**, with this:

```
None
@route
def run_cycle(
    state: State,
    cycles: str = "",
    cars_passing: int = 5
) -> Page:
    """Start a simulation or continue with saved settings."""
```

```
if cycles != "":
    if cycles not in ["5", "10"]:
        return Page(state, [
            "Choose 5 or 10 cycles.",
            Button("Back to Setup", "simulation_setup")
        ])

    if cars_passing < 1 or cars_passing > 100:
        return Page(state, [
            "Cars must be between 1 and 100.",
            Button("Back to Setup", "simulation_setup")
        ])

    # These lines must be outside the invalid-input condition.
    state.max_cycles = int(cycles)
    state.cars_passing = cars_passing
    state.current_cycle = 0
    state.traffic_history = [total_cars(state)]

if state.max_cycles == 0:
    return simulation_setup(state)

if state.current_cycle >= state.max_cycles:
    return simulation_results(state)

for direction in state.directions:
    direction.cars = direction.cars - state.cars_passing

    if direction.cars < 0:
        direction.cars = 0

state.current_cycle = state.current_cycle + 1
state.traffic_history.append(total_cars(state))
update_analysis(state)
```

```
    if state.current_cycle == state.max_cycles:
        return simulation_results(state)

    content = [
        "Cycle " + str(state.current_cycle)
        + " of " + str(state.max_cycles),
        "Cars passing per direction: " + str(state.cars_passing),
        "Total cars waiting: " + str(total_cars(state)),
        "Busiest direction: " + state.busiest_direction
    ]

    for description in traffic_summary(state):
        content.append(description)

    content.append(Button("Next Cycle", "run_cycle"))
    content.append(Button("View Summary", "simulation_results"))
    content.append(Button("Home", "index"))

    return Page(state, content)
```

Save and rerun. The simulation page will now display **“Cars passing per direction”** so you can see the setting it uses.

highlight exactly what needs to be changed

Only change the **beginning of `run_cycle`**.

**Replace this:**

```
None
@route
def run_cycle(state: State,cycles:str = "",cars_passing:int = 5) ->
Page:
    if cars_passing < 1 or cars_passing > 100:
```

```
        return Page(state, [
                "Cars  must be between 1-100.",
                Button("Back to Setup", "simulation_setup")])
        state.max_cycles = int(cycles)
        state.cars_passing = cars_passing
        state.current_cycle = 0
        state.traffic_history = [total_cars(state)]
```

**With this:**

```
None
@route
def run_cycle(state: State, cycles: str = "", cars_passing: int =
5) -> Page:
    """Start a simulation or continue with saved settings."""

    if cycles != "":                          # ADD this condition
        if cars_passing < 1 or cars_passing > 100:
            return Page(state, [
                "Cars must be between 1-100.",
                Button("Back to Setup", "simulation_setup")
            ])

        # These lines line up with the inner "if", NOT its
"return".
        state.max_cycles = int(cycles)
        state.cars_passing = cars_passing
        state.current_cycle = 0
        state.traffic_history = [total_cars(state)]
```

**Keep everything starting with this line unchanged:**

```
None
    if state.max_cycles == 0:
```

The change puts the saving code **outside the error condition**, so valid settings actually get saved.